Program Studi Teknik Informatika
Fakultas Teknologi Industri, Institut Teknologi Sumatera

IF25-11002 PRAKTIKUM PEMROGRAMAN

Modul Belajar 13

Fungsi dan Modularitas

Memecah program menjadi bagian bernama: parameter, nilai kembali, prototipe, dan referensi

Tim Pengajar
Pendamping Pertemuan 13. Tahun 2026

Komponen	Keterangan
Pertemuan	13 dari 16
Capaian	CPMK0607 (menerapkan) dan CPMK0608 (menjelaskan) pada konsep dasar algoritma dan sistem komputer
Waktu belajar	sekitar 3 jam belajar mandiri, ditambah 150 menit di kelas
Hands-on	`02 Hands-on/P13 - Fungsi - Hands-on/`

MODUL 13

Peta belajar

Ikuti urutan ini dari atas ke bawah. Setiap bagian memakai apa yang sudah dibahas di bagian sebelumnya.

Urutan	Bagian
1	Tentang modul ini
2	Masalah pembuka: Kode yang disalin berulang
3	Langkah 1: Fungsi void dan pemanggilannya
4	Langkah 2: Parameter dan nilai kembali
5	Langkah 3: Prototipe fungsi
6	Langkah 4: Pass by value dan by reference
7	Langkah 5: Array sebagai parameter
8	Langkah 6: Variabel lokal dan jangkauannya
9	Pesan error yang sering muncul
10	Latihan mandiri
11	Rangkuman
12	Cek pemahaman
13	Exit Ticket dan tugas
14	Bacaan lanjut dan persiapan minggu depan

PEMBUKA

Tentang modul ini

Modul ini mendampingi Pertemuan 13. Program kalian makin panjang: membaca data, mengurutkan, mencari, menampilkan. Menaruh semuanya di `main` membuat program sulit dibaca dan diperbaiki. Minggu ini kalian memecah program menjadi fungsi, bagian kecil bernama yang masing-masing mengerjakan satu tugas.

Isinya sama dengan slide di kelas, tetapi ditulis lebih pelan dan lebih lengkap, supaya kalian bisa mengulang sendiri di rumah tanpa harus menebak maksud slide.

Setelah menyelesaikan modul ini, kalian bisa

No	Kemampuan	Dibahas di
1	Menulis program yang dipecah menjadi fungsi dengan parameter dan nilai kembali yang tepat, termasuk fungsi `void`, fungsi `bool`, parameter referensi, dan parameter array (Sub-CPMK 13.1, CPMK0607)	Langkah 1 sampai 6
2	Menelusuri aliran nilai pada pemanggilan fungsi dan menjelaskan perbedaan pass by value dan pass by reference serta jangkauan variabel lokal (Sub-CPMK 13.2, CPMK0608)	kotak Tebak dulu, Latihan 3

Karena setiap instrumen penilaian dibagi rata antara Bagian A (menulis kode) dan Bagian B (menelusuri dan menjelaskan), yang dinilai bukan hanya program kalian jalan, tetapi juga kalian bisa menjelaskan mengapa program itu jalan.

Cara memakai modul ini

Setiap langkah punya pola yang sama: penjelasan konsep, kadang contoh dari kehidupan sehari-hari, lalu kode yang bisa langsung kalian ketik. Sesudah kode ada kotak **Tebak dulu**. Tulis tebakan kalian di kertas sebelum membaca **Jawaban** di bawahnya. Kebiasaan menebak lalu membuktikan inilah yang membuat konsep menempel, bukan sekadar membaca.

> PENTING **Ketik, jangan salin**
>
> Ketik ulang setiap contoh di GDB Online atau editor kalian, jangan disalin-tempel. Saat mengetik, kalian akan membuat salah ketik, membaca pesan error, dan memperbaikinya. Itu bagian dari belajar.

Yang perlu disiapkan

Peramban dengan akses ke onlinegdb.com, atau g++ di komputer kalian sendiri. Kertas dan alat tulis untuk menebak keluaran dan membuat trace table. Folder hands-on pertemuan ini dari repo mata kuliah.

PEMBUKA

Masalah pembuka: Kode yang disalin berulang

Program nilai kelas seorang asisten berisi kode mencari maksimum yang disalin empat kali: untuk kuis, tugas, UTS, dan UAS. Suatu hari ia menemukan kesalahan di kode itu.

Sekarang ia harus memperbaiki empat tempat, dan berharap tidak ada yang terlewat.

Tampilan yang diinginkan

```
Maks kuis  : 95
Maks tugas : 90
Maks UTS   : 88
Maks UAS   : 92
```

TEBAK DULU **Coba pikirkan**

Bagaimana caranya menulis kode mencari maksimum cukup sekali, lalu memakainya untuk empat array berbeda?

JAWABAN **Cara memecahnya**

Temukan yang berulang: Mencari maksimum dilakukan empat kali dengan data berbeda. Beri nama: Jadikan satu subprogram dengan masukan array dan keluaran nilai maksimum. Terjemahkan: Satu fungsi `int maksimum(int a[], int n)`, dipanggil empat kali.

LANGKAH 1

Fungsi void dan pemanggilannya

Fungsi adalah sekumpulan perintah yang diberi nama supaya bisa dipanggil berulang kali. Fungsi `void` mengerjakan sesuatu, misalnya menampilkan, tanpa memberikan nilai kembali.

Saat `garis(10)` dipanggil, nilai 10 disalin ke parameter `n`, badan fungsi dijalankan, lalu eksekusi kembali ke baris sesudah pemanggilan.

ANALOGI **Contoh sehari-hari**

Fungsi seperti resep. Resep ditulis sekali di buku, lalu dipakai berkali-kali dengan bahan berbeda. Bahan yang dibawa ke dapur adalah argumen; daftar bahan di resep adalah parameter.

Bentuk umum

```
void nama(tipe parameter) {
    // perintah
}

nama(argumen);   // memanggil
```

void.cpp

```
#include <iostream>
using namespace std;
void garis(int n) {
    for (int i = 0; i < n; i++) cout << "-";
    cout << endl;
}
int main() {
    garis(10);
    cout << "Laporan Nilai" << endl;
    garis(10);
    return 0;
}
```

TEBAK DULU

Sebelum menjalankan program, tulis keluarannya di kertas, karakter demi karakter.

JAWABAN

Keluaran

```
----------
Laporan Nilai
----------
```

`void` berarti fungsi tidak mengembalikan nilai. `n` adalah parameter; 10 adalah argumen.

LANGKAH 2

Parameter dan nilai kembali

Fungsi dengan tipe selain `void` memberikan nilai kembali lewat `return`. Pemanggilannya bisa dipakai di mana pun nilai bertipe itu boleh dipakai: disimpan ke variabel, ditampilkan, atau menjadi argumen fungsi lain.

`return` langsung mengakhiri fungsi. Pastikan setiap jalur di dalam fungsi berakhir dengan `return`; bila ada jalur yang tidak, g++ memberi peringatan dan hasilnya tidak bisa ditebak.

> ANALOGI **Contoh sehari-hari**
>
> Seperti memesan jus di kantin: kita memberi pesanan (argumen), penjual mengerjakannya, lalu menyerahkan segelas jus (nilai kembali).

Bentuk umum

```
tipe_kembali nama(tipe p1, tipe p2) {
    return hasil;
}
int m = nama(a, b);
```

return.cpp

```
double luasLingkaran(double r) {
    return 3.14 * r * r;
}
int lebihBesar(int a, int b) {
    return (a > b) ? a : b;
}
int main() {
    cout << luasLingkaran(10) << endl;
    int m = lebihBesar(7, 12);
    cout << m << " " << lebihBesar(m, 5) << endl;
    return 0;
}
```

> TEBAK DULU
>
> Sebelum menjalankan program, tulis keluarannya di kertas, karakter demi karakter.

JAWABAN

Keluaran

```
314
12 12
```

`return` mengirim nilai ke pemanggil dan langsung mengakhiri fungsi. Setiap jalur fungsi non-void harus berakhir dengan `return`.

LANGKAH 3

Prototipe fungsi

Compiler membaca dari atas ke bawah. Fungsi harus dikenalkan sebelum dipanggil. Ada dua cara: tulis definisinya di atas `main`, atau tulis prototipenya di atas dan definisinya di bawah. Prototipe membuat `main` tetap di atas dan mudah dibaca.

ANALOGI **Contoh sehari-hari**

Prototipe seperti daftar isi buku: memberi tahu bab apa saja yang ada, walaupun isinya baru muncul di halaman belakang.

Bentuk umum

```
bool genap(int x);   // prototipe, akhiri ;
int main() { ... genap(4) ... }
bool genap(int x) { ... }   // definisi
```

prototipe.cpp

```
#include <iostream>
using namespace std;
bool genap(int x);
int main() {
    for (int i = 1; i <= 4; i++) {
        bool g = genap(i);
        cout << i << (g ? " genap" : " ganjil") << endl;
    }
    return 0;
}
```

```
bool genap(int x) {
    return x % 2 == 0;
}
```

TEBAK DULU

Sebelum menjalankan program, tulis keluarannya di kertas, karakter demi karakter.

JAWABAN

Keluaran

```
1 ganjil
2 genap
3 ganjil
4 genap
```

Prototipe memberi tahu compiler nama, tipe kembali, dan parameter fungsi, sehingga definisinya boleh di bawah `main`.

LANGKAH 4

Pass by value dan by reference

Secara bawaan, argumen disalin ke parameter (pass by value). Perubahan di dalam fungsi hanya mengenai salinannya. Bila fungsi memang harus mengubah variabel pemanggil, misalnya fungsi tukar, gunakan parameter referensi dengan &.

ANALOGI **Contoh sehari-hari**

Memberi fotokopi dokumen kepada teman: coretan teman di fotokopi tidak mengubah dokumen asli kalian. Meminjamkan dokumen aslinya berbeda: coretannya langsung tertinggal di dokumen kalian.

Bentuk umum

```
void f(int x)    // salinan
void g(int &x)   // variabel aslinya
```

referensi.cpp

```cpp
void tambahNilai(int x) { x += 10; }
void tambahRef(int &x) { x += 10; }

int main() {
    int a = 5, b = 5;
    tambahNilai(a);
    tambahRef(b);
    cout << a << " " << b << endl;
    return 0;
}
```

TEBAK DULU

Sebelum menjalankan program, tulis keluarannya di kertas, karakter demi karakter.

JAWABAN

Keluaran

```
5 15
```

Tanpa &, fungsi menerima salinan. Dengan &, fungsi bekerja langsung pada variabel aslinya.

LANGKAH 5

Array sebagai parameter

Inilah jawaban masalah pembuka: satu fungsi `maksimum` dipakai untuk array berbeda. Fungsi tidak tahu panjang array, jadi panjangnya dikirim sebagai parameter kedua.

Berbeda dengan variabel biasa, array tidak disalin. Fungsi bekerja pada array aslinya, sehingga fungsi seperti `urutkan(a, n)` bisa mengubah isi array pemanggil.

ANALOGI **Contoh sehari-hari**

Seperti meminjamkan buku catatan asli ke teman, bukan fotokopinya: coretan teman langsung tertinggal di buku kita.

Bentuk umum

```
int maksimum(int a[], int n) { ... }
maksimum(data, 4);   // tanpa [ ]
```

arrayparam.cpp

```
int maksimum(int a[], int n) {
    int m = a[0];
    for (int i = 1; i < n; i++)
        if (a[i] > m) m = a[i];
    return m;
}
int main() {
    int kuis[4] = {80, 95, 70, 85};
    int uts[3] = {88, 76, 90};
    cout << maksimum(kuis, 4) << endl;
    cout << maksimum(uts, 3) << endl;
    return 0;
}
```

TEBAK DULU

Sebelum menjalankan program, tulis keluarannya di kertas, karakter demi karakter.

JAWABAN

Keluaran

```
95
90
```

Panjang array dikirim lewat parameter terpisah. Perubahan elemen di dalam fungsi mengenai array aslinya.

LANGKAH 6

Variabel lokal dan jangkauannya

Setiap variabel hanya bisa dipakai di bagian program tempat ia dideklarasikan (jangkauan atau scope). Variabel lokal sebuah fungsi lahir saat fungsi dipanggil dan hilang saat fungsi selesai. Karena itu dua fungsi boleh memakai nama variabel yang sama tanpa saling mengganggu.

> ANALOGI **Contoh sehari-hari**
>
> Seperti tamu acara: kartu tamu hanya berlaku di ruangan acara itu, dan tidak berlaku lagi setelah acara selesai.

Jenis	Dideklarasikan di	Hidup sampai	Contoh
Parameter	kurung fungsi	fungsi selesai	`n` pada `garis(int n)`
Lokal fungsi	badan fungsi	fungsi selesai	`m` pada `maksimum`
Lokal blok	di dalam `{ }`	blok selesai	`i` pada `for`

> PENTING
>
> Variabel `i` di `main` dan `i` di fungsi lain adalah dua variabel berbeda, walaupun namanya sama.

WASPADA

Pesan error yang sering muncul

Semua pesan di tabel ini dihasilkan g++ dengan opsi `-Wall`, sama seperti yang dipakai `cek.sh`.

Kesalahan	Contoh	Pesan pertama dari g++
Fungsi dipanggil sebelum dikenal	`panggil sebelum definisi`	`error: 'kuadrat' was not declared in this scope`
Tidak ada return di semua jalur	`int f() tanpa return akhir`	`warning: control reaches end of non-void function`
Banyak argumen tidak cocok	`lebihBesar(1, 2, 3)`	`error: too many arguments to function 'int lebihBesar(int, int)'`
Literal ke parameter referensi	`tukar(a, 5)`	`error: cannot bind non-const lvalue reference of type 'int&' to an rvalue of ...`
Nilai void dipakai	`int x = garis(5);`	`error: void value not ignored as it ought to be`

Peringatan pada baris kedua patut dianggap serius. Fungsi yang tidak mengembalikan nilai di sebuah jalur memberi hasil acak, dan `cek.sh` menolaknya karena memakai `-Werror`.

LATIHAN

Latihan mandiri

Latihan ini sama dengan hands-on di kelas. Semua file ada di `02 Hands-on/P13 - Fungsi - Hands-on/latihan/`. Kerjakan berurutan, lalu cocokkan dengan `expected/` atau jalankan `bash cek.sh`.

No	File	Tingkat	Yang dilatih
1	`handson1-bangun.cpp`	Dasar	fungsi `void` dan fungsi dengan `return`
2	`handson2-prima.cpp`	Menengah	fungsi `bool`, prototipe, array sebagai parameter
3	`handson3-referensi.cpp`	Tantangan	telusuri parameter, perbaiki tiga kesalahan fungsi
+	`bonus-statistik.cpp`	Pengayaan	pustaka fungsi untuk array

Latihan 1: handson1-bangun.cpp

Tiga fungsi bangun datar dan garis, dipanggil dari `main`.

Latihan 2: handson2-prima.cpp

Fungsi `isPrima` dan `hitungPrima` dengan prototipe di atas `main`.

Latihan 3: handson3-referensi.cpp

Tiga fungsi dengan kesalahan pass by value, pembagian bulat, dan jalur tanpa `return`. Telusuri, perbaiki, dan jelaskan.

Bonus: bonus-statistik.cpp

Fungsi minimum, maksimum, rata-rata, dan pengurutan untuk satu array.

Latihan menelusuri

tebak.cpp

```
int f(int x, int &y) {
    x = x * 2;
    y = y + x;
    return x + y;
}
```

```
int main() {
    int a = 3, b = 4;
    int c = f(a, b);
    cout << a << " " << b << " " << c << endl;
    return 0;
}
```

TEBAK DULU

Tulis keluarannya di kertas tanpa menjalankan program. Buat trace table bila perlu.

JAWABAN

Keluaran

```
3 10 16
```

`x` salinan dari `a` menjadi 6, tetapi `a` tetap 3. `y` adalah referensi ke `b`, jadi `b` menjadi 4 + 6 = 10. Nilai kembali 6 + 10 = 16.

PENUTUP

Rangkuman

Konsep	Yang perlu diingat
Fungsi void dan pemanggilannya	`void` berarti fungsi tidak mengembalikan nilai.
Parameter dan nilai kembali	`return` mengirim nilai ke pemanggil dan langsung mengakhiri fungsi.
Prototipe fungsi	Prototipe memberi tahu compiler nama, tipe kembali, dan parameter fungsi, sehingga definisinya boleh di bawah `main`.
Pass by value dan by reference	Tanpa `&`, fungsi menerima salinan.
Array sebagai parameter	Panjang array dikirim lewat parameter terpisah.
Variabel lokal dan jangkauannya	Variabel `i` di `main` dan `i` di fungsi lain adalah dua variabel berbeda, walaupun namanya sama.

PENUTUP

Cek pemahaman

Jawab tanpa menjalankan program, lalu periksa dengan menjalankannya.

1. Apa beda parameter dan argumen?
2. Mengapa `void tukar(int x, int y)` tidak menukar variabel pemanggil?
3. Apa gunanya prototipe fungsi?
4. Apakah array di dalam fungsi adalah salinan?
5. Apa yang terjadi pada variabel lokal sesudah fungsi selesai?

JAWABAN

1. Parameter adalah nama di definisi fungsi; argumen adalah nilai yang dikirim saat memanggil.
2. Parameternya salinan; perlu referensi `int &x, int &y`.
3. Mengenalkan fungsi ke compiler sehingga definisinya boleh ditulis sesudah `main`.
4. Bukan; fungsi bekerja pada array aslinya.
5. Variabel itu hilang dan tidak bisa dipakai lagi.

PENILAIAN

Exit Ticket 13

Dikerjakan di 25 menit terakhir kelas, sendiri, tanpa AI, internet, atau diskusi. Exit Ticket 13 adalah satu dari 14 Exit Ticket; rata-ratanya menjadi komponen berbobot 25%.

Soal	Isi	Poin	CPMK
A1	Tulis fungsi yang mengembalikan nilai terbesar dari tiga bilangan	25	CPMK0607
A2	Tulis fungsi `void` yang mengubah array menjadi kuadrat setiap elemennya	25	CPMK0607
B1	Telusuri nilai variabel sebelum dan sesudah pemanggilan fungsi dengan referensi	20	CPMK0608
B2	Jelaskan mengapa sebuah fungsi tukar tidak bekerja	20	CPMK0608
B3	Jelaskan jangkauan variabel pada potongan kode yang diberikan	10	CPMK0608

PENILAIAN

Tugas 13: Library Matematika

Kumpulkan sebelum Pertemuan 14 lewat kanal pengumpulan yang diumumkan dosen.

Bagian A: program (50 poin)

Buat program berisi kumpulan fungsi matematika berikut dengan prototipe di atas `main`, lalu buat menu interaktif untuk memilih dan menguji setiap fungsi dengan validasi masukan.

1. `int faktorial(int n)` dan `int pangkat(int basis, int eksponen)`
2. `bool isPrima(int n)`
3. `int gcd(int a, int b)` dan `int lcm(int a, int b)`
4. `double luasSegitiga(double alas, double tinggi)` dan `double kelilingLingkaran(double r)`
5. `void tampilkanDeret(int n)` yang mencetak 1 sampai n
6. Menu berulang dengan pilihan keluar

> ANALOGI **Bonus, tidak wajib**
>
> Tambahkan fungsi `void tukar(int &a, int &b)` dan tunjukkan bedanya dengan versi tanpa `&`.

Bagian B: penjelasan (50 poin)

Komentar maksimal 150 kata: telusuri pemanggilan `lcm(4, 6)` sampai ke `gcd`, jelaskan kapan memakai `void` dan kapan memakai tipe kembali, dan beri satu contoh perbedaan pass by value dan pass by reference dari program kalian.

Rubrik Tugas 13

Kriteria	Poin	Sangat baik 85–100	Baik 70–84	Cukup 55–69	Kurang <55
A1 Program berjalan	20	tanpa error dan peringatan	ada peringatan	perlu satu perbaikan	tidak terkompilasi
A2 Kelengkapan fungsi	15	delapan fungsi dan menu	satu fungsi kurang	dua kurang	tiga atau lebih kurang
A3 Ketepatan dan validasi	15	semua hasil tepat dan masukan divalidasi	satu salah	dua salah	sebagian besar salah
B1 Penelusuran pemanggilan	30	jejak lcm ke gcd tepat, alasan jelas	satu langkah keliru	tanpa jejak	tidak ada atau disalin
B2 Cerita error dan pengujian	20	pesan, sebab, perbaikan, uji	pesan tidak dikutip	hanya perbaikan	tidak ada

Nilai kriteria = poin × skor level. Contoh: A1 di level Baik dengan skor 80 memberi 20 × 0,80 = 16 poin.

PENUTUP

Bacaan lanjut dan persiapan minggu depan

1. Deitel, P. dan Deitel, H. C++ How to Program, edisi ke-10, Bab 6.
2. learncpp.com, Bab 2 dan 12 (functions, references).
3. Gaddis, T. Starting Out with C++, Bab 6.
4. Slide `01 Slide/P13 Fungsi dan Modularitas.pdf` dan hands-on di `02 Hands-on/P13 - Fungsi - Hands-on/`.

> PENTING **Minggu depan**
> Pertemuan 14 membahas rekursi, yaitu fungsi yang memanggil dirinya sendiri, dan struct untuk mengelompokkan data berbeda tipe.